Mind the Gap? A Comparison of Pre-Training Datasets for Tabular Foundation Models

Ano N. Ymous

Abstract

Tabular foundation models are pre-trained on one of three classes of corpus: curated datasets drawn from benchmark repositories, tables harvested at scale from the web, or synthetic tables sampled from a parametric generative prior. Despite the centrality of pre-training data to model performance, little is known about how these corpora relate to one another in distribution. In this work we take three canonical, archetypal datasets used to train tabular foundation models; the T4 dataset represents web-scraped corpuses, the TabFM dataset curated tables from Kaggle, and the TabICL dataset as a well-used synthetic prior with publically available parameters. We characterise each corpus using aggregate features over whole tables, columns and correlations, and compare them using discriminator AUCs and k-NN coverage metrics. We find that the TabICL synthetic prior occupies a narrow region of the space of real tables, that this mismatch cannot be closed by optimising prior hyper-parameters across more than 86 thousand configurations, and that curated and web-scraped corpora are broadly interchangeable in feature space. Surprisingly, the distributional gap between synthetic pre-training data and real tables has no measurable effect on downstream in-context learning performance, suggesting that coverage of the real-data distribution is not the primary driver of TFM generalisation.

1 Introduction

Tabular data is among the most prevalent data modalities in applied machine learning, spanning domains from healthcare and finance to scientific research and industry [21, 2, 15, 23]. Unlike images or text, tabular data lacks a canonical structure, with datasets varying widely in the number of columns, the mix of numeric and categorical features, and the statistical properties of individual columns. This heterogeneity has historically made it difficult to transfer knowledge across datasets, and classical approaches typically require training a new model from scratch for each task.

Tabular foundation models (TFMs) offer a promising alternative. By pre-training on large corpora of tabular data, TFMs can perform in-context learning (ICL) at inference time — generalising to new tasks from a small number of examples without further gradient updates [20, 14, 25, 26]. This paradigm has shown competitive performance against task-specific baselines [14, 26], but its success depends critically on the quality and diversity of the pre-training corpus.

Three broad strategies have emerged for constructing these corpora. The first collates tables from existing benchmark repositories, as in TabFM [25]. The second harvests tables at scale from the web, exemplified by T4 and Tabula-8B [12]. The third forgoes real data entirely, instead sampling synthetic tables from a parametric generative prior; TabICL [20] and TabPFN [14] follow this approach, using structural causal models to generate diverse classification tasks.

Despite the centrality of pre-training data to TFM performance, surprisingly little is known about how these three corpora relate to one another. Do curated and web-scraped datasets occupy similar regions of the space of possible tables? How well do synthetic priors, designed without reference to real data, approximate the distributions found in practice? If there is a mismatch, can the parameters

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

of the prior be tuned to close it? And what impact does proximity to real distributions actually have over downstream performance?

1.1 Research Questions

We structure this work around the following sequential research questions:

RQ1: Do synthetic priors cover real tables? Synthetic priors are designed using the causal processes that produce real tables, but whether they actually do is an open question. We show that synthetic priors occupy a very small region of the space occupied by real tables, but that the space of synthetic tables is well-covered by real tables.

RQ2: Do curated datasets cover web-scraped tables? Curated datasets make use of tables known to be 'clean' and useful for ML algorithms, and here we show that web-scraped tables broadly cover curated tables and vice-versa, explaining why pre-training on comparatively few tables yields comparable performance.

RQ3: Can synthetic priors be optimised to cover real tables? Given that neither web-scraped or curated datasets are well-covered by synthetic priors, the question remains as to whether the same prior design can be parameterised to match these real tables. Here we show that multiple methods of optimisation yield little improvement in coverage, with near perfect separation of synthetic and real tables after a highly extensive (86k trial) grid search.

RQ4: Does proximity to synthetic prior impact performance? Regardless of how well synthetic priors match real world tables and distributions, an essential question is whether proximity to the prior actually impacts downstream performance. We present the preliminary finding that, under our feature sets, performance shows no clear correlation with proximity to the prior.

2 Background & Related Work

We denote a table as t with $|R|$ rows and $|C|$ columns. Column j is written C_j, with cardinality $||C_j||$ denoting the number of unique values. The uniqueness ratio $\kappa_j = ||C_j||/|R|$ gives the proportion of unique values; we treat columns with $\kappa < 0.2$ as categorical.

2.1 Curated Datasets

Curated datasets are assembled by curating tables from established machine learning repositories such as OpenML [24] and the UCI Machine Learning Repository [3] or the competition platform Kaggle[1]. Rather than collecting new data, this approach aggregates existing benchmarks into a unified corpus, applying filters for quality, schema consistency, and licence compatibility. TabFM [25] is the primary example in this setting, comprising approximately 380 datasets drawn from Kaggle after deduplication and normalisation of column types.

A known limitation of curated corpora is that they inherit the biases of the repositories from which they are drawn. Certain domains, particularly medical and financial datasets, are heavily overrepresented, while the distribution of dataset sizes skews small, reflecting the historical tendency of benchmark repositories to favour datasets amenable to classical ML methods [18]. These biases may limit the diversity of structural patterns seen during pre-training.

2.2 Web-Scraped Corpuses

An alternative to manual curation is to harvest tabular data at scale from the web. The T4 dataset [12] and its respective TFM Tabula-8B follow this approach, extracting tables from HTML pages and other structured web sources to produce corpora orders of magnitude larger than hand-curated alternatives. The principal advantage is diversity: web-scraped corpora capture a wide range of domains, table sizes, and column types that are unlikely to appear in curated benchmark repositories.

This scale comes at a cost. Web-scraped tables are noisy, often containing malformed schemas, merged header rows, or semantically inconsistent columns. Licencing is also ambiguous, as the

[1] `https://www.kaggle.com/`

provenance of individual tables is difficult to trace [12]. Pre-processing pipelines therefore play a critical role in determining the effective quality of the resulting corpus.

2.3 Synthetic Priors

A third paradigm avoids real data entirely, instead sampling tables from a parametric generative prior. This approach offers several practical advantages: there are no licensing constraints, data can be generated at arbitrary scale, and the distribution can be controlled directly via the prior's hyperparameters. Early work in this vein includes the priors used in Prior-Fitted Networks (PFN) and TabPFN [14], which generate synthetic classification tasks from simple Bayesian network structures.

TabICL [20] extends this idea using structural causal models (SCMs) as the generative mechanism. A SCM defines a directed acyclic graph over variables, with each node's value determined by a learned or sampled function of its parents plus noise. TabICL parameterises this process via a hyperparameter vector $\theta \in \theta$ that governs choices such as the SCM type, the tree model family used to instantiate causal mechanisms, class balance, and the probability of generating categorical columns. Sampling from the prior thus yields synthetic tables whose statistical properties are determined by θ.

The fundamental limitation of synthetic priors is distributional mismatch: there is no guarantee that tables sampled from the prior resemble real-world tabular data in their structural or distributional properties. Whether this mismatch is consequential for downstream ICL performance, and whether it can be reduced by tuning θ, is the central question of this paper.

3 Method

In this work we investigate, according to our research questions, the degree to which web-scraped, curated and synthetic prior tabular datasets cover the same spaces. We conduct this through two primary metrics:

Discriminators: Samples that broadly cover the same space should be indistinguishable to a given model. With this in mind, over the feature space described below, we fit discriminators to predict which class of dataset a given table belongs to. This assesses the realism of samples on an individual basis, and is common in comparing sets of samples in generative modelling [13, 4].

Coverage: A more direct distance-based metric, here we make use of neighbours-based coverage metrics [22, 17, 19] to assess the degree to which these tabular datasets overlap in a given feature space. These metrics are commonly applied in generative modelling, assessing how well a given model produces samples that cover the original data distribution.

3.1 Features

Given the challenges of operating over datasets of tables (column and row order invariance, heterogenous column types, etc) it is necessary to move tables into a unified feature space. We construct a set of simple aggregate features that summarise table, column and inter-column patterns and distributions. Features are, with the exception of column count, invariant to the number of columns and rows in a table, and in our later optimisation sections the column count becomes effectively redundant. Further detail on these features can be found in Appendix A.1, and a discussion of their simplicity in Section 6.

We perform limited ablation over this feature set to ensure robustness in our results, using the following feature sets:

- **Full** ($d = 70$): for table-level analysis, the complete feature vector $\phi(t)$.
- **Scalars** ($d = 9$): for table-level analysis, distributional scalar features, excluding the number of columns.
- **Histograms** ($d = 60$): for table-level analysis, average histogram mean and standard deviation features only.
- **Col. Hists** ($d = 50$)for column-level analysis, cumulative column histograms extracted from individual tables.

- **Corr. Hists** ($d = 50$) for inter-column level analysis, correlation histograms over tables.

These ablations and varied feature sets allow us to validate the robustness of our findings, showing that our findings are neither specific to only one feature set, nor reliant on one single feature.

3.2 Discriminator

Given a set of real tables $\mathcal{T}_r$ and synthetic tables $\mathcal{T}_s$, we train a binary discriminator to distinguish between the two populations. Each table t is embedded as $\phi(t) \in \mathbb{R}^{70}$ using the above-described features and assigned label $y = 1$ (real) or $y = 0$ (synthetic), yielding dataset

$$\mathcal{D} = \{(\phi(t),\, 1) : t \in \mathcal{T}_r\} \cup \{(\phi(t),\, 0) : t \in \mathcal{T}_s\}. \tag{1}$$

Given the number of necessary trials, concessions are made for speed and efficiency. We use a XGBoost classifiers trained on stratified splits of $\mathcal{D}$. A value of AUC $= 0.5$ indicates the discriminator performs at chance, with synthetic and real tables indistinguishable, while AUC $= 1.0$ indicates perfect separability. We therefore use AUC as a proxy for *prior realism*, and all prior optimisation methods minimise it.

3.3 Coverage

As well as using a discriminator as a measure of *sample* realism, we use coverage as a measure of distributional overlap. This is a set of methods and metrics developed primarily in generative modelling. Sajjadi et al. [22] first frame 'recall' and 'precision' for generative models as a measure of distributional coverage. Kynkäänniemi et al. [17] then extend this work through manifold estimation, with [19] proposing k-NN-based metrics along this theme which are highly related to the metrics we describe below.

We measure the fraction of the real distribution not covered by the synthetic prior via a k-nearest-neighbour estimator, as well as the inverse, the fraction of the synthetic distribution not covered by the real. Feature vectors are normalised to $[0, 1]$. A coverage threshold $\delta_{0.95}$ is set as the 95th percentile of mean within-synthetic k-NN distances at $k = 5$. A real table is considered *uncovered* if its mean distance to its k nearest synthetic neighbours exceeds this threshold:

$$f_{\text{unc}} = \frac{1}{|\mathcal{T}_r|} \sum_j \mathbf{1}\left[\bar{d}_k(r_j,\, \mathcal{T}_s) > \delta_{0.95}\right]. \tag{2}$$

Coverage: *A point p is* covered *by a set Q, written $p \in Q$, if its mean distance to its k nearest neighbours in Q does not exceed $\delta_{0.95}$. For two sets P and Q, we write $P \in Q$ to denote the proportion of points in P covered by Q:*

$$P \in Q \;=\; \frac{1}{|P|} \sum_{p \in P} \mathbf{1}[p \in Q]. \tag{3}$$

Recall: *The* Recall *of Q with respect to P is the dataset-level coverage of P by Q. In our setting Recall measures the proportion of real tables covered by the synthetic prior.*

Precision: *The* Precision *of Q with respect to P is the dataset-level coverage of Q by P. In our setting Precision measures the proportion of synthetic tables covered by the real distribution.*

3.4 Datasets

Curated, web-scraped and synthetic prior-based datasets all aim to include enough statistical patterns that a TFM can downstream be applied to an arbitrary tabular dataset, the intent being that some of the downstream patterns will have appeared during pre-training. To this extent, it is fair to say that the intent of these pre-training datasets is to *cover* the space of downstream tables.

Curated and Web-Scraped First we select representative examples of web-scraped and curated datasets of real-world tables. As a web-scraped dataset we use the **T4** corpus from Gardner et al. [11]. This dataset was used to pre-train the model Tabula-8B, a language-model based TFM, and is a trimmed version of the massive TabLib [9] corpus, sourced from GitHub and CommonCrawl. As

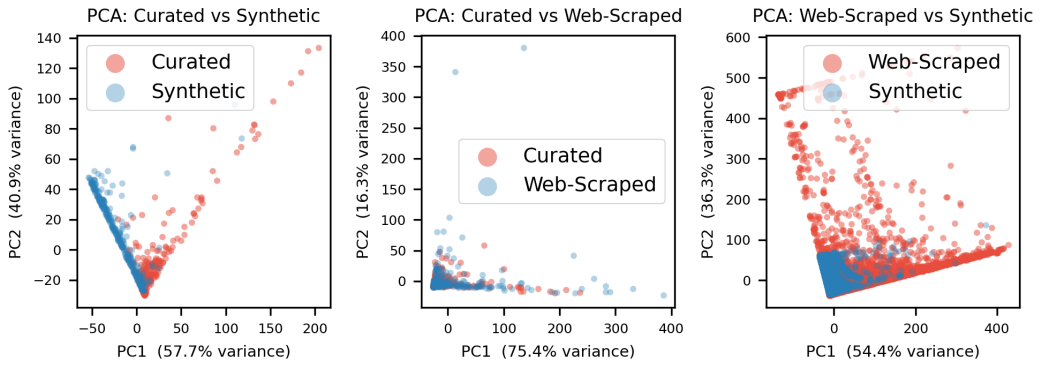


Figure 1: PCA projections of the curated FM dataset, the web-scraped T4 dataset, and the synthetic ICL datasets within our feature space. Sample count is equal for each pair.

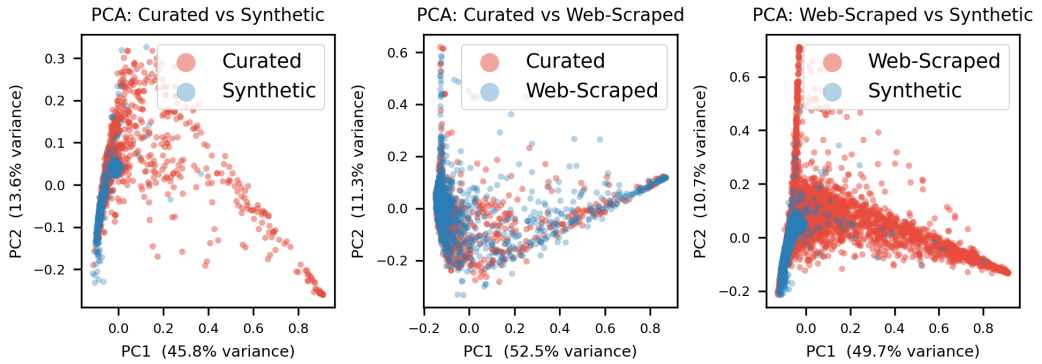


Figure 2: PCA projections of cumulative histograms for the curated FM dataset, the web-scraped T4 dataset, and the synthetic ICL datasets within our feature space. Sample count is equal for each pair.

a curated dataset, we use the collection of Kaggle datasets presented in Wen et al. [25], 384 mixed classification and regression datasets selected by various properties (e.g. row and column count, completeness). This dataset was used to train the TFM TabFM; where appropriate we refer to the dataset by the moniker **FM**.

Synthetic Prior It remains to select a reasonable prior framework for generating tables. We use that from TabICL[20], which builds on the successes of the TabPFN [14] prior, and performs very strongly on benchmarks when compared with other single-prior methods. We study a single prior, TabICL, for two reasons: first, it is the only canonical synthetic prior whose default parameters and generation code are fully open-source (TabPFN does not release prior parameters; Mitra does not distribute data-level code); second, restricting to one prior makes the 86k-configuration optimisation in Section 5 computationally tractable.

A notable pattern in TFM development is closed-source code – Mitra [26] does not distribute data-level code, and TabPFN [14] do not share their prior parameters. Comparisons without access to default parameters is simply not possible. Despite this, TabICL functions in a highly similar manner to TabPFN and other priors, including mixture models like Mitra, all of which rely on SCM-based modelling.

As a result, TabICL's priors are a best-case, with both strong, canonical performance and default parameters and code both open-source. TabICL, within the literature of synthetic priors, builds on the successes of other priors, meaning that our findings may also carry over to these and other similar priors should their parameters become available. We encourage future research to release open-source parameters so that this analysis can be extended.

4 Comparisons Between Datasets

In this section we address our first two research questions through treating datasets as-is, without optimisation towards either dataset of real tables. In Figure 1, we show PCA projections of the feature sets for each dataset plotted against each other Figures showing cumulative coverage curves, feature importances and aggregates of our histogram features are available in the Appendix. We present our results for discriminators and coverage in Table 1, including validation of our discriminator and coverage metrics over two independent 1000-size samples of the respective dataset.

Dataset				**Bootstrap AUC**	**Coverage**	
A	**B**	**Features**	**Rank**	**Mean ± Std**	**Recall**	**Precision**
Validation: Self-Comparisons Between Subsets						
T4				0.498 ± 0.015	0.940	0.957
ICL		Full		0.501 ± 0.037	0.944	0.963
FM				0.500 ± 0.009	0.938	0.955
Unoptimised: Synthetic vs. Web-scraped (RQ1)						
ICL	T4	Full		0.9997 ± 0.0001	0.0882	0.9880
		Histograms		0.9985 ± 0.0003	0.1617	0.9849
		Scalars		0.9991 ± 0.0002	0.0854	0.9318
		Col. Hists		0.977 ± 0.003	0.250	0.987
		Corr. Hists		0.988 ± 0.011	0.187	1.00
Unoptimised: Synthetic vs. Curated (RQ1)						
ICL	FM	Full		0.9972 ± 0.0029	0.1604	0.9780
		Histograms		0.9943 ± 0.0049	0.2421	0.8742
		Scalars		0.9958 ± 0.0037	0.2736	0.6132
		Col. Hists		0.972 ± 0.007	0.421	0.970
		Corr. Hists		0.983 ± 0.014	0.276	0.992
Unoptimised: Web-scraped vs. Curated (RQ2)						
T4	FM	Full		0.9133 ± 0.0237	0.9748	0.8302
		Histograms		0.8884 ± 0.0276	0.9748	0.8836
		Scalars		0.8972 ± 0.0250	0.9906	0.8805
		Col. Hists		0.906 ± 0.013	0.968	0.852
		Corr. Hists		0.841 ± 0.052	0.951	0.902
Grid search optimised (AUC), ICL vs. T4 — top-5 ranked configurations (RQ3)						
ICL	T4	Full	1	0.9989 ± 0.0013	0.063	0.996
			2	0.9987 ± 0.0014	0.080	0.992
			3	0.9985 ± 0.0016	0.125	0.991
Bayesian optimised (Equation 4), ICL vs. real datasets — best run per dataset (RQ3)						
ICL	FM	Full	1	0.995 ± 0.018	0.194	0.981
	T4		1	0.997 ± 0.006	0.218	0.978

Table 1: Bootstrap AUC and two-directional coverage across all experiments. The upper section shows unoptimised pairwise comparisons grouped by research question. The middle section shows the top-5 grid-search configurations (AUC-optimised) evaluated against T4. The lower section shows the best Bayesian-optimised configurations (joint AUC + coverage loss, Equation 4) validated at larger sample sizes. Recall = B covered by A (A∈B); Precision = A covered by B (B∈A).

4.1 RQ1: Real vs. Synthetic

Discriminator AUCs between ICL and our datasets of real tables are consistently high, within-margin of perfect performance when considering deviations during bootstrapping (200 re-samples). Coverage is similarly very low between ICL and our real tables, and comparatively high in reverse, indicating that the real tables cover the synthetic far more effectively than the inverse.

Feature importances indicate that histogram features prove the most useful for each pairwise discriminator (Figure 5). On comparing synthetic and real datasets, and with consideration of our visualisation in Figure 6, we note that at intermediate histogram bins ICL columns are consistently lower than columns from real tables, and have low variance within that region, making the middle range of column distributions highly informative for discriminators. Discriminators remain highly performant when histogram features are included, indicating that simple summary statistics (in-particular the number of columns and the skewness of distributions) are still highly informative.

We also investigate the same metrics on column and inter-relation level distributions. Over columns we use cumulative histograms, and over correlations simple histograms. These results are reported in Table 1 and visualised through PCA in Figure 2. Examples of cumulative histograms over columns can be found in Figure 9.

Column and correlation AUCs are somewhat lower than for the whole feature set, due to decreased expressiveness compared to our full features which compare both. Recall is also higher, though still comparably low, again due to the limited expressiveness of these feature sets. That being said, AUCs are still near-perfect for Real/Synthetic discriminators, with recall consistently below 50%.

From these results we can conclude that with default parameters, ICL priors produce tables with un-realistic patterns within columns. From coverage analysis, we can conclude that the distribution of synthetic tables is a narrow region of the space which real tables occupy. Further, from column and correlation histograms as features, it is clear that these patterns hold true both within columns and in how columns relate to one another.

4.2 RQ2: Curated vs. Web-Scraped

Here we see significantly lower discriminator AUCs than for synthetic and real tables, though performance is still strong. Coverages are higher in both directions, with T4 covering almost all of the FM tables, and FM covering the large majority ($> 83\%$) of T4 tables. While histogram features still prove the most informative, it is notable that none of them contribute a gain of more than 6%, indicating a more challenging classification task for discriminators within this feature space.

Given these results we can conclude that while curated tables are less easily distinguished from web-scraped, the individual column values are still significantly different. That said, both occupy very similar spaces, with high coverages in both directions showing near-overlap between the two distributions within our feature space.

5 RQ3: Optimisation of Priors

Given the findings of the previous section, namely that the published parameters for the TabICL priors do not well-cover the space of real tables, in this section we explore whether the prior can instead be optimised to better cover these spaces. We optimise AUC to avoid conflation between loss function and final coverage metric.

The TabICL generative prior is parameterised $\theta \in \theta$ governing structural choices such as SCM type, tree model family, class balance, and the probability of generating categorical columns. We seek $\theta^* = \text{argmin}_{\theta \in \theta}\ \text{AUC}(\theta)$, where $\text{AUC}(\theta)$ is the discriminator score when $N_{\text{eval}} = 200$ synthetic tables are drawn from the prior parameterised by θ. Three search strategies are evaluated: Bayesian optimisation, a genetic algorithm, and a structured grid search. The search space, i.e. the parameters for the TabICL prior over which we attempt to optimise, is outlined in the Appendix. We make the trivial optimisation of sampling column counts from the distribution of columns counts in the respective real dataset, meaning that this can be effectively removed from our search space.

5.1 Bayesian, Genetic Searches - AUC

We begin our attempts at optimisation using bayesian and genetic optimisation approached.

Bayesian optimisation We use the Tree-structured Parzen Estimator (TPE) as implemented in Optuna [1]. At each trial, TPE fits two kernel density estimates over previously evaluated configurations: $\ell(\theta)$ over the top-performing quartile and $g(\theta)$ over the remainder. The next candidate maximises the expected improvement proxy $\ell(\theta)/g(\theta)$. We run $N_{\text{trials}} = 1000$ trials per source dataset.

Genetic algorithm A population of $N_{\text{pop}} = 16$ candidate configurations is evolved over $N_{\text{gen}} = 100$ generations. Each generation proceeds follows the process outlined in the relevant Appendix section.

5.1.1 Results

Neither the Bayesian nor genetic optimisation is able to significantly decrease the AUC of a discriminator, despite extensive experiments and attempts at finding superior optimisation performance. Though both methods have limited guarantees on convergence, their failure does suggest that the prior cannot produce tables that match those from T4. To ensure this finding is not a product of our optimisation processes, we conduct a high-resolution grid search over the prior space.

5.2 Grid Search - AUC

We construct a discrete grid by sampling each continuous or integer parameter at G evenly-spaced values between its bounds, and taking the Cartesian product. Conditional parameters are only included where relevant, reducing the search space substantially. At our grid resolution of $G = 6$, chosen to remain somewhat computationally tractable, this results in more than 86 thousand prior parameters and evaluations. Even with only 200 real and synthetic samples, this is still highly computationally expensive. This analysis was parallelised over a cluster of 6 nodes, each with 2 NVIDIA Grace CPU Superchips, and took a little less than 23 hours.

Figure 8 shows AUCs over the whole grid search. The lowest recorded AUC over this grid search is still > 0.95, with the large majority not significantly different from the un-optimised synthetic prior. Given the computational cost of this search we limit dataset sizes to 200 for both T4 and FM datasets, but perform subsequent analysis with top-ranked parameter combinations over 1000 samples to ensure robustness. To avoid overfitting we reduce our discriminator trees to 100.

These results are presented in Table 1 for the 5 highest ranked parameter sets. Validation over our larger sample sizes leads to the resumption of near-perfect AUC, and coverage metrics are near-consistent with the unoptimised prior. Taken together, this search has shown that the ICL prior simply cannot produce consistently realistic samples.

Validation - AUC Following the grid search, the top-$K = 5$ configurations by AUC are subjected to a deeper post-hoc evaluation. For each selected configuration, $N_{\text{synth}} = 1{,}000$ synthetic tables are generated and their feature vectors $\phi(t) \in \mathbb{R}^{70}$ extracted. These are compared against an equal-sized random subsample of the T4 tables. Five analyses are run per configuration.

5.3 Bayesian Search - Coverage + AUC

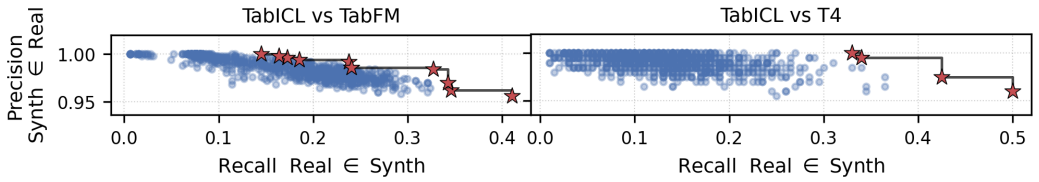


Figure 3: Recall and Precision for optimising the ICL prior to match the FM and T4 datasets, towards the loss function defined in Equation 4. The Recall/Precision Pareto front is marked with red stars.

Given that all optimisations over AUC failed, we also experiment with a loss function that includes coverage alongside AUC through a Euclidean combination as in Equation 4:

$$\mathbf{L} = \left(1/\sqrt{3}\right) \cdot \sqrt{(A \in B)^2 + (B \in A)^2 + (2 \cdot (0.5 - AUC))^2} \tag{4}$$

A grid search is less tractable here, as coverage is expensive to calculate compared to AUC, so we limit our methods to a Bayesian search. We use 50 Bayesian optimisers, each of 20 trials, as higher trial counts did not lead to improved convergence in our analysis over AUC. We also use larger evaluation counts of 500 prior-generated tables to reduce aleatoric noise. A visualisation of Recall and Precision for each trial step can be found in Figure 3.

We find that while optimising this three-term loss function does increase recall (Real $\in$ Synth), AUC remains high, and all three metrics degrade when re-run with the same prior parameters and larger sample sizes for the best-performing runs (see Table 1). We note that there is a Pareto front along the Recall/Precision metrics; the best runs have higher recall only with the penality of reduced precision.

6 Discussion

Here we discuss two key points related to our above findings. First, whether our simple aggregate feature sets are adequate for our findings, and secondly, what material impact our findings have on model performance downstream.

6.1 Choice of Features

In this work we make use of simple, aggregate feature sets. Should we aim to argue that we have properly optimised our synthetic prior to match real data, it would be necessary to demonstrate that our feature set adequately captures the data to make such a claim. In our case, however, it is necessary only to demonstrate that along some given feature space, the original table spaces are easily distinguishable. If the tables are distinguishable under a simple feature set, they will similarly be distinguishable under more expressive feature sets, and therefore it is not necessary to exhaust the space of possible expressions of the data. The same logic applies to our geometric analysis with Recall and Precision.

It is worth noting, however, that our numeric results *are specific to this feature set*. All comparisons should be phrased as *within this feature space*; there likely are other feature expressions of this data which would yield lower AUC and higher Recall. This does not reduce the validity of our work – it is only necessary to find one or a few feature sets to justify our findings. In other words, while another feature set might show high Recall, we have demonstrated through these features that the ICL prior does not cover real tables.

6.2 RQ4: Proximity to Prior and Performance

Despite the results presented above — namely that the ICL prior can likely not be optimised to cover, and produce realistic samples from, the space of real tables — it remains to investigate whether this gap has a detectable relationship with downstream TFM performance under our feature-based proximity measures. In most machine-learning settings we might expect that the further a model is applied from its training distribution, the lower its performance becomes compared to other models.

To this end, we assemble a collection of classical models (See Appendix C) along with TabICL. We then apply this set of models on classification datasets from the TALENT benchmarks [18] and record the mean rank of ICL. The TALENT benchmarks are specifically curated to reflect a large span of potential downstream applications, and are used in-literature to reflect the validity of an approach [20, 15]. Our expectation, as in most machine-learning settings, is that increasing distance from the prior distribution leads to decreased ranking. Having collected these rankings, we calculate 5-NN distance using **(i)** Our full, table-level feature set **(ii)** Cumulative column histograms **(iii)** Correlation histograms over the table. We visualise this experiment in Figure 4.

There is no clear pattern between mean ICL rank and a table's proximity to the ICL prior distribution, and correlations are all below 0.31. This is an unintuitive finding – as previously discussed, we would typically expect that proximity to the pre-training distribution leads to greater comparative performance. Together with RQ1–RQ3, this is consistent with the interpretation that synthetic prior pre-training is substantially out-of-distribution, yet this alone does not appear to determine downstream performance —- at least for TabICL and the proximity measures considered here.

The mechanism for this is not well-understood. One possible interpretation is that in-context learning is inherently more robust to pre-training distributional shift than standard supervised learning. Alternatively, the inductive biases instilled by SCM-based pre-training may constitute a form of regularisation that transfers broadly across real tables regardless of low-level distributional alignment. The fairly flat ranking of ICL with k-NN distance, including at the tail at extreme distances, suggests the former is more applicable, but stronger conclusions are difficult to draw.

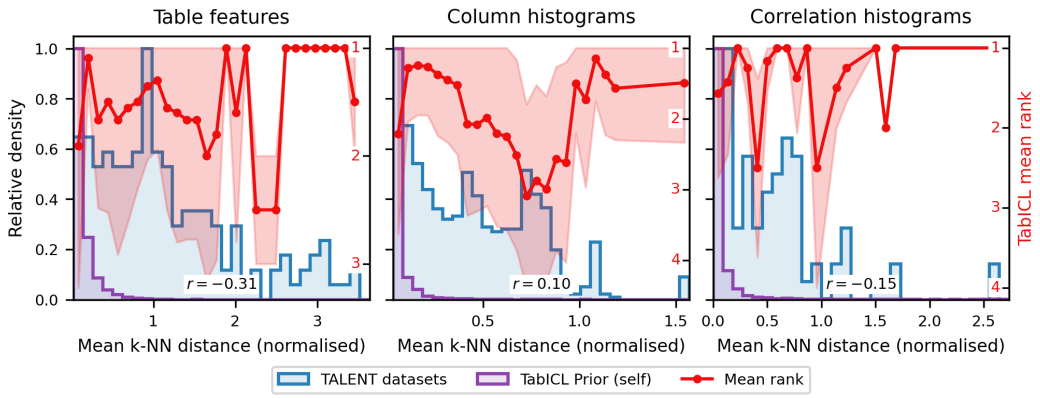


Figure 4: k-NN histograms (5 neighbours) calculated for ICL against itself and the TALENT benchmark tables across our full feature set, cumulative column histograms, and correlation histograms. Plotted alongside is the mean TabICL rank against classical benchmarks for the tables within that bin.

Further stratification of these results by individual table-level features would largely recapitulate information already expressed through aggregate proximity to the prior, which is itself a function of those same underlying features. Directly testing the two candidate mechanisms through pre-training under alternative or degraded priors is left to future work due to computational constraints. The consistency of the null result across three distinct feature representations, with each capturing different aspects of table structure, constitutes meaningful evidence that distributional coverage is not a primary driver of TFM generalisation, independent of a complete mechanistic account. Distinguishing between these accounts, and understanding more generally under what conditions OOD pre-training is benign or even beneficial, remains an open question that we leave to future work.

7 Conclusion

We have investigated the distributional relationships between the three principal classes of pre-training corpora for tabular foundation models. These corpora are selected from among the limited set of available datasets as highly representative of their approaches. Our findings across three research questions are as follows.

The archetypal TabICL prior occupies a narrow and unrealistic region of the space of real tables. Discriminators achieve near-perfect separation between ICL and both real datasets under all feature sets, and coverage of real tables by the synthetic prior is consistently below 25%. Extensive attempts to close this gap through optimisation of the prior hyperparameters θ yield no meaningful improvement. The lowest AUC recorded across more than 86 thousand grid evaluations remains above 95%, and coverage-aware optimisation similarly degrades at larger sample sizes. As such the distributional mismatch between the TabICL prior and real tabular data is not a tuning problem but a structural limitation of this prior. Whether this extends to all SCM-based priors, or to structurally different synthetic generation mechanisms, remains open pending the release of other priors' parameters.

Curated and web-scraped datasets, by contrast, occupy broadly overlapping regions of the feature space. T4 covers nearly all of the curated FM tables, and FM covers the large majority of T4, providing a principled explanation for the empirical observation that pre-training on comparatively few curated datasets can match web-scale performance.

Unintuitively, we find no evidence that the distributional gap between synthetic pre-training data and real tables, as captured by our feature sets, predicts downstream ICL performance for the model studied. TabICL's mean rank against classical baselines shows no correlation with a test table's proximity to the ICL prior, across any of the feature sets examined. This suggests that TFMs pre-trained on synthetic data are not succeeding by interpolating within their training distribution in the conventional sense, and that the inductive biases instilled by synthetic pre-training may matter more than distributional coverage. The mechanism underlying this phenomenon is not well-understood, and we leave its investigation to future work.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, pages 2623–2631, New York, NY, USA, July 2019. Association for Computing Machinery. ISBN 978-1-4503-6201-6. doi: 10.1145/3292500.3330701. URL https://dl.acm.org/doi/10.1145/3292500.3330701.

[2] L. Akoglu, M. McGlohon, and C. Faloutsos. RTM: Laws and a recursive generator for weighted time-evolving graphs. In *Proceedings - IEEE international conference on data mining, ICDM*, pages 701–706, 2008. ISBN 978-0-7695-3502-9. doi: 10.1109/ICDM.2008.123.

[3] S. D. Bay, D. Kibler, M. J. Pazzani, and P. Smyth. The UCI KDD archive of large data sets for data mining research and experimentation. *SIGKDD Explor. Newsl.*, 2(2):81–85, Dec. 2000. ISSN 1931-0145. doi: 10.1145/380995.381030. URL https://dl.acm.org/doi/10.1145/380995.381030.

[4] A. Borji. Pros and cons of GAN evaluation measures. *Computer Vision and Image Understanding*, 179:41–65, Feb. 2019. ISSN 1077-3142. doi: 10.1016/j.cviu.2018.10.009. URL https://www.sciencedirect.com/science/article/pii/S1077314218304272.

[5] L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, Oct. 2001. ISSN 1573-0565. doi: 10.1023/A:1010933404324. URL https://doi.org/10.1023/A:1010933404324.

[6] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 785–794, New York, NY, USA, Aug. 2016. Association for Computing Machinery. ISBN 978-1-4503-4232-2. doi: 10.1145/2939672.2939785. URL https://dl.acm.org/doi/10.1145/2939672.2939785.

[7] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, Sept. 1995. ISSN 1573-0565. doi: 10.1007/BF00994018. URL https://doi.org/10.1007/BF00994018.

[8] A. V. Dorogush, V. Ershov, and A. Gulin. CatBoost: gradient boosting with categorical features support, Oct. 2018. URL http://arxiv.org/abs/1810.11363. arXiv:1810.11363 [cs].

[9] G. Eggert, K. Huo, M. Biven, and J. Waugh. TabLib: A Dataset of 627M Tables with Context, Oct. 2023. URL http://arxiv.org/abs/2310.07875. arXiv:2310.07875 [cs].

[10] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data, Mar. 2020. URL http://arxiv.org/abs/2003.06505. arXiv:2003.06505 [stat].

[11] J. Gardner, J. C. Perdomo, and L. Schmidt. Large Scale Transfer Learning for Tabular Data via Language Modeling, Nov. 2024. URL http://arxiv.org/abs/2406.12031. arXiv:2406.12031 [cs].

[12] J. Gardner, J. C. Perdomo, and L. Schmidt. Large Scale Transfer Learning for Tabular Data via Language Modeling. In *Advances in Neural Information Processing Systems*, volume 37, pages 45155–45205, Dec. 2024. doi: 10.52202/079017-1435. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/4fd5cfd2e31bebbccfa5ffa354c04bdc-Abstract-Conference.html.

[13] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html.

[14] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, Jan. 2025. ISSN 1476-4687. doi: 10.1038/s41586-024-08328-6. URL https://www.nature.com/articles/s41586-024-08328-6. Publisher: Nature Publishing Group.

[15] J.-P. Jiang, S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, and H.-J. Ye. Representation Learning for Tabular Data: A Comprehensive Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–20, 2026. ISSN 1939-3539. doi: 10.1109/TPAMI.2026.3657217. URL https://ieeexplore.ieee.org/abstract/document/11369258.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: a highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pages 3149–3157, Red Hook, NY, USA, Dec. 2017. Curran Associates Inc. ISBN 978-1-5108-6096-4. URL https://dl.acm.org/doi/10.5555/3294996.3295074.

[17] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila. Improved Precision and Recall Metric for Assessing Generative Models. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper/2019/hash/0234c510bc6d908b28c70ff313743079-Abstract.html.

[18] S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, H.-H. Yin, T. Zhou, J.-P. Jiang, and H.-J. Ye. Talent: A Tabular Analytics and Learning Toolbox. *Journal of Machine Learning Research*, 26(226):1–16, 2025. ISSN 1533-7928. URL http://jmlr.org/papers/v26/25-0512.html.

[19] M. F. Naeem, S. J. Oh, Y. Uh, Y. Choi, and J. Yoo. Reliable Fidelity and Diversity Metrics for Generative Models. In *Proceedings of the 37th International Conference on Machine Learning*, pages 7176–7185. PMLR, Nov. 2020. URL https://proceedings.mlr.press/v119/naeem20a.html.

[20] J. Qu, D. Holzmüller, G. Varoquaux, and M. L. Morvan. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 50817–50847. PMLR, Oct. 2025. URL https://proceedings.mlr.press/v267/qu25d.html.

[21] H. O. Quinn, M. Sedky, J. Francis, and M. Streeton. Literature Review of Explainable Tabular Data Analysis. *Electronics*, 13(19), Sept. 2024. ISSN 2079-9292. doi: 10.3390/electronics13193806. URL https://www.mdpi.com/2079-9292/13/19/3806.

[22] M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet, and S. Gelly. Assessing Generative Models via Precision and Recall. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper/2018/hash/f7696a9b362ac5a51c3dc8f098b73923-Abstract.html.

[23] R. Shwartz-Ziv and A. Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, May 2022. ISSN 1566-2535. doi: 10.1016/j.inffus.2021.11.011. URL https://www.sciencedirect.com/science/article/pii/S1566253521002360.

[24] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo. OpenML: networked science in machine learning. *SIGKDD Explor. Newsl.*, 15(2):49–60, June 2014. ISSN 1931-0145. doi: 10.1145/2641190.2641198. URL https://dl.acm.org/doi/10.1145/2641190.2641198.

[25] X. Wen, H. Zhang, S. Zheng, W. Xu, and J. Bian. From Supervised to Generative: A Novel Paradigm for Tabular Deep Learning with Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, pages 3323–3333, New York, NY, USA, Aug. 2024. Association for Computing Machinery. ISBN 979-8-4007-0490-1. doi: 10.1145/3637528.3671975. URL https://dl.acm.org/doi/10.1145/3637528.3671975.

[26] X. Zhang, D. C. Maddix, J. Yin, N. Erickson, A. F. Ansari, B. Han, S. Zhang, L. Akoglu, C. Faloutsos, M. W. Mahoney, C. Hu, H. Rangwala, G. Karypis, and B. Wang. Mitra: Mixed Synthetic Priors for Enhancing Tabular Foundation Models, Oct. 2025. URL http://arxiv.org/abs/2510.21204. arXiv:2510.21204 [cs].

A Comparisons Between Datasets

A.1 Features

We begin with simple aggregate features over table structure:

Num Cols: The dimensionalities $|R|$ and $|C|$ of the table

Categorical Ratio The proportion of columns with $\kappa < 0.2$, inferred as categorical under the uniqueness heuristic.

$$\frac{1}{|C|}\sum_{j=1}^{|C|}\mathbf{1}[\kappa_j < 0.2]\,. \tag{5}$$

A.1.1 Within-Column

We then compute a set of features to express properties of the numeric or categorical values within columns, which are then aggregated over the table. Let $\mathcal{C}_{\text{num}} = \{j : \kappa_j \geq 0.2\}$ denote the set of inferred numeric columns, with per-column mean $\mu_j = \mathbb{E}[C_j]$ and standard deviation σ_j.

Mean cardinality for categorical columns: Let $\mathcal{C}_{\text{cat}} = \{j : \kappa_j < 0.2\}$. The mean number of distinct values across inferred categorical columns, $\bar{\mathcal{C}_{\text{cat}}}$.

$$\frac{1}{|\mathcal{C}_{\text{cat}}|}\sum_{j\in\mathcal{C}_{\text{cat}}}\|C_j\|. \tag{6}$$

Max cardinality for categorical columns: The maximum number of distinct values across inferred categorical columns.

Skewness: The third standardised moment of each numeric column, averaged across all columns. We summarise across columns via the mean $\overline{\text{skew}} = \frac{1}{|\mathcal{C}_{\text{num}}|}\sum_j \text{skew}(C_j)$ and standard deviation σ_{skew}.

Kurtosis: The fourth standardised moment (excess), averaged across all columns.

Entropy for categorical columns: The Shannon entropy of the empirical value distribution of each inferred categorical column.

A.1.2 Between-Column

Also necessary to express is how columns relate to one another; this inter-column structure is captured via the Pearson correlation matrix over numeric columns. Let $\mathbf{P} \in \mathbb{R}^{|\mathcal{C}_{\text{num}}|\times|\mathcal{C}_{\text{num}}|}$ be the matrix of pairwise Pearson correlations, and let $\mathcal{P} = \{|P_{jk}| : j \neq k\}$ denote the multiset of off-diagonal absolute correlations.

We extract three summary statistics; the mean absolute correlation $\bar{\rho} = (1/|\mathcal{P}|)\cdot\sum_{r\in\mathcal{P}} r$, the standard deviation of absolute correlations σ_ρ, and the proportion of highly correlated pairs $r < 0.7$.

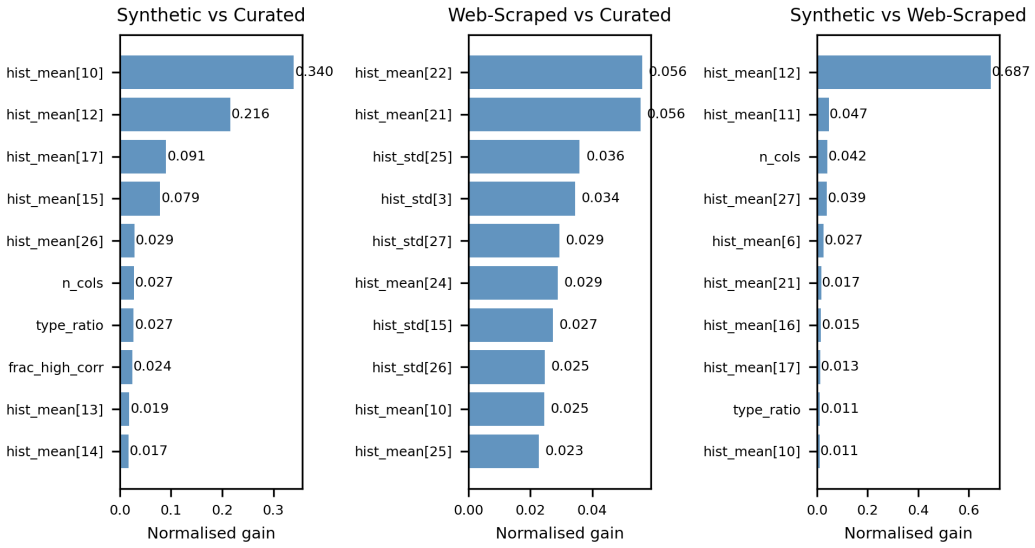


Figure 5: Feature importances, assessed from performance gains through tree splits, for each pairwise discriminator over our three datasets.

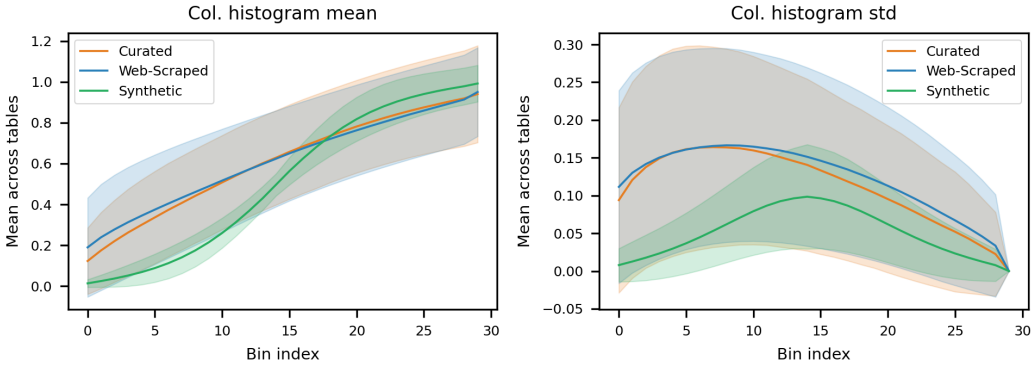


Figure 6: Example visualisations of our aggregate column histogram features. Left: Mean bin values across tables. Right: Mean bin deviations across tables.

B Optimisation

B.1 Search Space θ

The search space θ is defined over the following parameters, partitioned by their conditional relevance to the SCM prior type τ:

Shared (active for all τ):

- $\lambda_c \in \{2, \ldots, 20\}$ — maximum number of target classes.
- $p_{\text{cat}} \in [0,\ 0.6]$ — probability of generating a categorical column.
- $p_{\text{ord}} \in [0,\ 1]$ — probability of ordered multiclass targets.
- *balanced* $\in \{0, 1\}$ — whether class balance is enforced.
- *replay_small* $\in \{0, 1\}$ — whether small-dataset replay is used.
- $\tau \in \{\texttt{mlp_scm},\ \texttt{tree_scm},\ \texttt{mix_scm}\}$ — SCM prior type.

Conditional on $\tau \in \{\texttt{tree_scm},\ \texttt{mix_scm}\}$:

- $m \in \{\texttt{DT},\ \texttt{ET},\ \texttt{RF},\ \texttt{XGB}\}$ — tree model family.
- $\lambda_d \in [0.3,\ 1.0]$ — tree depth rate parameter.
- $\lambda_n \in [0.3,\ 1.0]$ — number-of-estimators rate parameter.

Conditional on $\tau = \texttt{mix_scm}$ only:

- $p_{\text{mlp}} \in [0,\ 1]$ — MLP mixing probability (the tree weight is $1 - p_{\text{mlp}}$).

Two additional parameters are fixed per dataset rather than searched: the column-count bounds $(\min_f, \max_f)$, derived from the range of $|C|$ observed across the cached real-table features.

B.2 Genetic Algorithm

1. **Evaluation**: compute AUC(θ_i) for all i.
2. **Elitism**: retain the $N_{\text{elite}} = 4$ fittest individuals unchanged.
3. **Selection**: fill remaining slots via tournament selection with $k = 3$ competitors; the individual with the lowest AUC wins.
4. **Crossover**: pairs of selected parents produce offspring by sampling each parameter uniformly from either parent.
5. **Mutation**: each parameter is independently mutated with probability $p_{\text{mut}} = 0.5$. Continuous parameters receive Gaussian noise $\mathcal{N}(0,\ 0.1 \cdot (\text{hi} - \text{lo}))$; integer parameters use $\mathcal{N}(0,\ (\text{hi} - \text{lo})/6)$; categorical parameters are resampled uniformly.

B.3 Further Figures

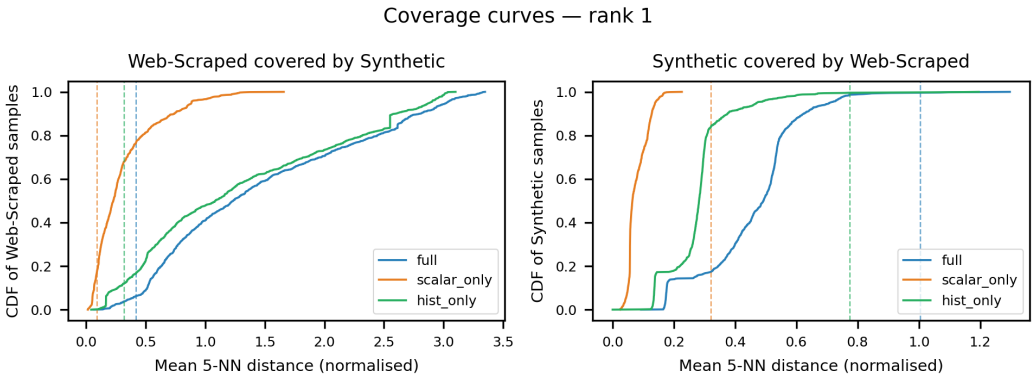


Figure 7: Cumulative coverage curves for ICL over T4 (left) and T4 over ICL (right).

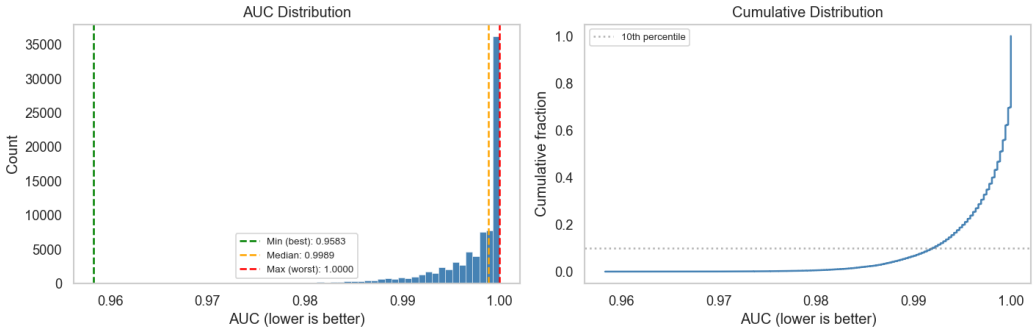


Figure 8: A histogram and a cumulative histogram of AUC values over our grid search.

B.4 Column Examples

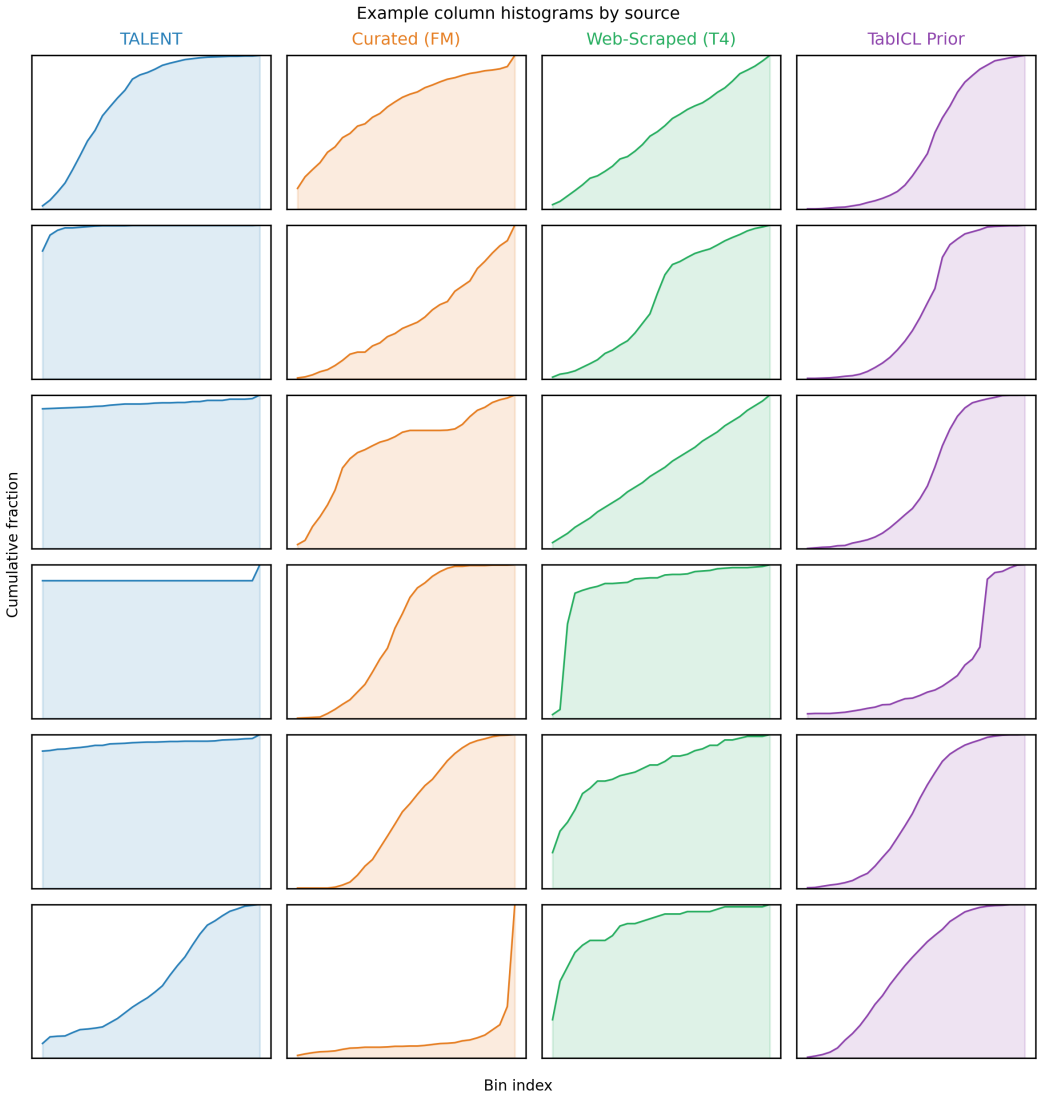


Figure 9: Examples of cumulative histograms over columns randomly selected from the TALENT benchmarks, the curated FM dataset, the web-scraped T4 dataset, and the synthetic prior-based ICL dataset.

C Benchmarks: Classical Modelss

We benchmark TabICL against a suite of classical and automated machine learning baselines on all binary and multi-class classification datasets in the TALENT benchmark [18]. Each model is evaluated under identical preprocessing and, where applicable, averaged over 3 random seeds. All models receive the full training data. For TabICL this means that the maximum amount of training data is presented as context.

C.1 Gradient-Boosted Trees

We include three gradient-boosted decision tree methods: XGBoost [6], LightGBM [16], and CatBoost [8]. All three build an additive ensemble of shallow trees by greedily minimising a regularised loss, differing primarily in their tree-growth strategy and regularisation scheme. Each ensemble contains up to 2 000 trees, with early stopping applied on a held-out validation split to prevent overfitting.

C.2 Random Forest

We include a Random Forest [5] of 2 000 trees. Unlike boosting methods, each tree is trained independently on a bootstrap sample of the data with a random subset of features considered at each split, and predictions are aggregated by majority vote. Trees are capped at depth 12.

C.3 Logistic Regression

Logistic Regression with ℓ_2 regularisation serves as a linear baseline. It models the log-odds of each class as a linear function of the input features, making it well-suited to linearly separable problems and providing a simple reference point for non-linear methods.

C.4 Support Vector Machine

We include a Support Vector Machine [7] with an RBF kernel, which implicitly maps inputs to a high-dimensional feature space and finds the maximum-margin separating hyperplane therein. SVM is particularly competitive on small, high-dimensional datasets.

C.5 k-Nearest Neighbours

k-Nearest Neighbours (k-NN) classifies each test point by majority vote among its $k = 5$ nearest neighbours in the training set, weighted by inverse distance. It is a non-parametric method with no explicit training phase, making it a useful lower bound on the difficulty of each dataset.

C.6 AutoGluon

AutoGluon [10] is an automated machine learning system that constructs a multi-layer stacked ensemble, combining gradient-boosted trees, neural networks, and other base learners selected and weighted automatically. It is included as a strong, well-tuned ensemble baseline that is representative of the current practical state of the art for tabular classification.

D Benchmarking: Alternative Performance Metrics

As well as the ranking analysis presented in Figure 4, we substitute ranking for both the TabICL AUC relative to the mean benchmark performance:

$$r_{\text{AUC}} = \frac{\text{AUC}_{\text{ICL}}}{\sum_{i \in \text{Benchmarks}} \text{AUC}_{\text{i}}} \tag{7}$$

and the TabICL AUC relative to the best performing benchmark:

$$r'_{\text{AUC}} = \frac{\text{AUC}_{\text{ICL}}}{\max\left(\{\text{AUC}_{\text{i}} | i \in \text{Benchmarks}\}\right)}. \tag{8}$$

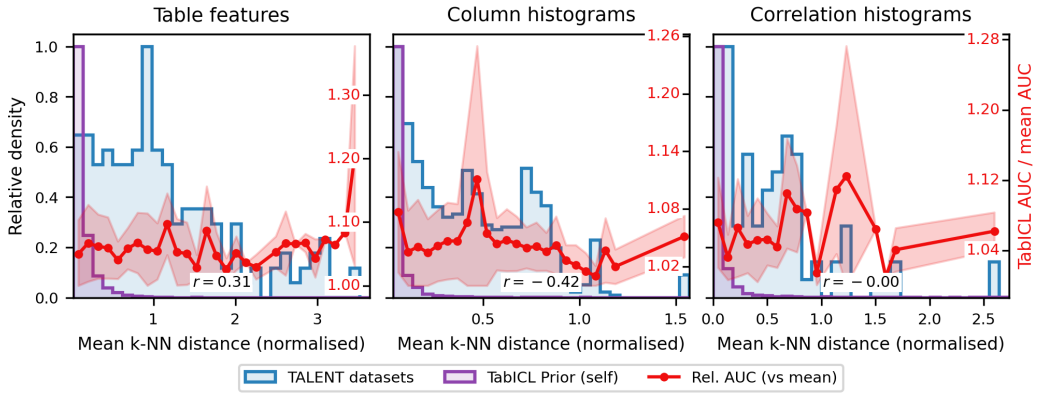


Figure 10: k-NN histograms (5 neighbours) calculated for ICL against itself and the TALENT benchmark tables across our full feature set, cumulative column histograms, and correlation histograms. Plotted alongside is the mean TabICL rank against classical benchmarks for the tables within that bin.

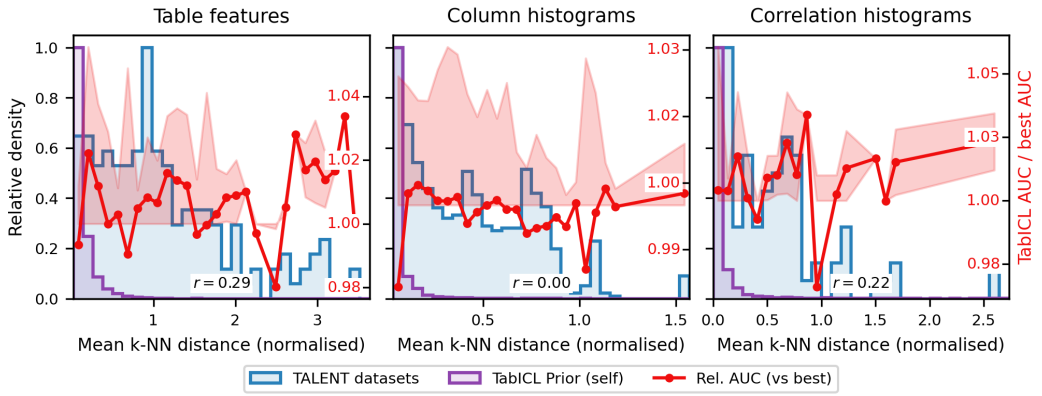


Figure 11: k-NN histograms (5 neighbours) calculated for ICL against itself and the TALENT benchmark tables across our full feature set, cumulative column histograms, and correlation histograms. Plotted alongside is the mean TabICL rank against classical benchmarks for the tables within that bin.

The mean relative AUC metric is shown in Figure 10, and AUC relative to benchmark maximum in Figure 11. Here we see the same lack of pattern, and low correlations, that we observed when using the rank of the TabICL compared to the benchmark. These alternative metrics therefore corroborate our findings in the main text.